

White Paper

Bryan Alexander Buchorn
Independent Researcher
bryan.alexander@buchorn.com

Abstract—This module forms a critical component of the CEREBRUM framework, a community-structured graph attention engine for Knowledge Graph reasoning. It focuses on Framework Implementation Guide.

Index Terms—Knowledge Graph, Graph Attention, DSCF, CSA, Hierarchical Reasoning.

CEREBRUM: Community-Structured Graph Attention for Knowledge Graph Reasoning

A White Paper

Authors: Bryan Alexander Buchorn · Claude Sonnet 4.6 (Research Collaborator)

Date: March 2026

Status: **Version 1.1.0 · Phase 20 COMPLETE — 994 tests passing**

License: Proprietary — all rights reserved

Abstract

We propose CEREBRUM, a novel framework that enables Knowledge Graphs (KGs)

to perform multi-hop reasoning using the same structural principles that make

Transformer-based Large Language Models powerful — without requiring an LLM,

without training data, and with full interpretability of every inference step.

The central contribution is Community-Structured Attention (CSA): a

mechanism in which graph communities serve as attention heads, graph traversal

replaces matrix multiplication, and hop depth replaces layer depth. Unlike

Graph Attention Networks (GATs), which apply learned attention within hard

adjacency constraints, CSA uses community membership as a soft global

constraint that captures both local topological cohesion and global structural

significance simultaneously.

This is made possible by a second contribution: the Dual-Signal Community

Fusion (DSCF) algorithm, which produces communities that encode both LPA

majority-vote structure (local) and modularity gain (global) in a single

partition. We show that DSCF communities possess a structural duality that

maps naturally to the dual character of multi-head attention in Transformers.

Together, CSA and DSCF form an architecture where a KG can answer multi-hop

questions by traversing itself, with every reasoning step grounded in explicit

graph edges, every conclusion traceable to a path, and no LLM required for

inference — though one may optionally be used for natural language generation.

Version 0.3.x extends the core architecture with production hardening (JWT authentication, ResourceGovernor, asynchronous streaming traversal), a full real-time streaming subsystem (five signal discretizer classes, sliding-window buffer with reference-counted edge eviction, incremental ego-network community updates, SSE streaming API), experience-dependent structural relay formation via Bridge Twin Nodes (Phase 12), and directional causal edge inference via Spike-Timing Dependent Plasticity (Phase 13). The core CSA and DSCF algorithms are unchanged; all extensions sit above them.

0.1 Acknowledgments: Intellectual Debt and Credits

CEREBRUM stands on the shoulders of decades of research in graph theory, community detection, and neural networks. We explicitly acknowledge the foundational work of the following researchers and the algorithms that form the bedrock of our framework:

- 1) **LPA (Label Propagation Algorithm):** Usha Nandini Raghavan, Réka Albert, and Shailesh Kumara (2007). Their work on near-linear time community detection via local neighbor voting provided the "Local Signal" for our DSCF engine.
- 2) **Louvain Algorithm:** Vincent Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre (2008). Their greedy modularity optimization method established the global structural baseline for community detection.
- 3) **Leiden Algorithm:** Vincent Traag, Ludo Waltman, and Nees Jan van Eck (2019). Their refinement of Louvain, ensuring internal connectivity, provides the "Global Signal" and connectivity post-pass for DSCF.

- 4) **Graph Attention Networks (GATs)**: Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio (2018). Their introduction of learned attention on graphs served as the primary foil and inspiration for our Community-Structured Attention (CSA).
- 5) **KG Embeddings (TransE / RotatE)**: Antoine Bordes et al. (2013) and Zhiqing Sun et al. (2019). Their work on representing relational knowledge in vector spaces provides the semantic grounding layer for CSA.
- 6) **GraphRAG**: Microsoft Research / Edge et al. (2024). Their pioneering work in combining community summaries with LLM retrieval provided the immediate context and competitive baseline for CEREBRUM's grounded reasoning approach.
- 7) **Avionics Engineering**: The concept of **mid-level voting** (or mid-value selection) in triplex-redundant aircraft navigation. This engineering principle of multi-sensor consensus served as the foundational inspiration for the multi-signal logic of DSCF and TSC.
- 8) **PageRank**: Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank Citation Ranking: Bringing Order to the Web. Stanford InfoLab. Used as the global authority prior in CSA and as a THALAMUS structural encoding feature.
- 9) **Betweenness Centrality**: Freeman, L.C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1), 35–41. Used as a THALAMUS positional encoding feature.
- 10) **Simulated Annealing**: Kirkpatrick, S., Gelatt, C.D., & Vecchi, M.P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. The DSCF temperature decay schedule is a direct application.
- 11) **Bloom Filters**: Bloom, B.H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7), 422–426. Used in HolographicIndex for federated blind graph discovery.
- 12) **Spike-Timing Dependent Plasticity (STDP)**: Bi, G.Q. & Poo, M.M. (1998). *Journal of Neuroscience*, 18(24), 10464–10472. Markram, H. et al. (1997). *Science*, 275(5297), 213–215. The STDPDiscretizer and Bridge Twin formation are direct computational analogs.
- 13) **Hebbian Learning**: Hebb, D.O. (1949). *The Organization of Behavior*. Wiley. The Bridge Twin LTP/LTD analog and InsightEngine reward propagation are grounded in this principle.
- 14) **Beam Search**: Lowerre, B.T. (1976). The HARPY Speech Recognition System. PhD thesis, Carnegie Mellon University. BeamTraversal is a direct application to graph path expansion.
- 15) **Sentence-BERT**: Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*. Default embedding backend when [embeddings] extra is installed.

1) Introduction

1.1 The Gap Between Knowledge Graphs and Language Models

Knowledge Graphs and Large Language Models represent two fundamentally different approaches to knowledge representation and reasoning.

Knowledge Graphs store knowledge explicitly: entities as nodes, relationships

as typed edges, facts as (subject, predicate, object) triples.

This makes

them precise, verifiable, and updatable without retraining.

However, they

cannot reason beyond what is explicitly stored. Multi-hop inference — "Marie

Curie discovered Polonium, Polonium is radioactive, therefore Marie Curie

discovered a radioactive element" — requires either

hardcoded graph traversal

queries or external reasoning systems.

Large Language Models store knowledge implicitly in billions of weight

parameters. They can reason, generalize, and synthesize

across domains. But

this implicit representation is opaque, cannot be updated without expensive

fine-tuning, and is prone to hallucination — generating

plausible-sounding

but incorrect facts with no mechanism for ground-truth verification.

The field has responded with hybrid approaches: Retrieval-Augmented Generation

(RAG), Knowledge-Graph-augmented LLMs, and

GraphRAG. In all of these, the KG

is a retrieval store and the LLM does the reasoning. The

KG remains passive.

CEREBRUM inverts this relationship. The KG reasons.

The LLM, if present,

only generates natural language from the KG's output.

Every inference step is

a graph traversal, every conclusion is a path, and every

path can be verified.

1.2 The Core Observation

A Transformer's power comes from three mechanisms working together:

Multi-head attention: different heads specialize on different

relational aspects of the input (syntactic, semantic, long-range, etc.)

Deep composition: each layer builds on the previous, allowing complex

multi-step reasoning

Positional awareness: the model knows where each token sits relative

to others

We observe that Knowledge Graphs have natural analogs for all three:

Community structure serves the role of attention heads: nodes within

a community have strong mutual relevance; communities specialize on

different conceptual domains.

BFS hop depth serves the role of layer depth: each hop is one step of composed reasoning.

Graph-structural features (PageRank, betweenness, degree) serve the role of positional encoding: they tell the model where each entity sits in the global information landscape.

The question is: can these analogs be made operational — not merely metaphorical? We argue yes, and demonstrate the architecture to do so.

1.3 System Component Names

The CEREBRUM stack uses the following named layers, reflecting both computational role and biological analogy:

CEREBRUM — the overarching product/framework name.

THALAMUS — the ingestion engine: pluggable adapters, EmbeddingEngine, StructuralEncoder, and STDPDiscretizer. Named for the thalamus, which receives and preprocesses all sensory input before routing it to the cortex.

CORTEX — the reasoning engine: DSCF community detection, CSA attention formula, BeamTraversal, and AnswerExtractor. Named for the cortex, where structured reasoning occurs.

REM Engine — graph self-reorganization: pruning low-confidence edges, re-running community detection, and synthesizing new hypothesis edges. Analogous to REM sleep and hippocampal memory consolidation.

Bridge Twin Engine — experience-dependent structural relay node formation. Analogous to thalamic relay nuclei and LTP/LTD synaptic plasticity.

1.4 Contributions

This paper makes the following primary contributions:

The CEREBRUM architecture: a complete mapping of Transformer components

to KG operations, organized into THALAMUS (ingestion) and CORTEX (reasoning) layers, enabling multi-hop reasoning via graph traversal alone.

Community-Structured Attention (CSA): a novel attention mechanism using

community membership as a soft global constraint on graph traversal,

bridging the gap between local GAT-style attention and global Transformer-style attention.

Dual-Signal Community Fusion (DSCF): a novel community detection

algorithm combining LPA majority-vote and modularity gain simultaneously

at each node update, producing communities with dual short-range/long-range

character that maps to multi-head attention's dual specialization.

1) Background and Related Work

2.1 Graph Neural Networks

Graph Neural Networks (GNNs) [Scarselli et al., 2009] generalize neural

networks to graph-structured data. The message-passing paradigm [Gilmer et

al., 2017] defines node updates as aggregations of neighbor representations.

Graph Attention Networks (GATs) [Velickovic et al., 2018] introduce learned

attention weights between connected nodes. Key limitations for our purposes:

(a) attention is restricted to direct neighbors — no global context;

(b) communities are not considered; (c) training labels required.

GraphSAGE [Hamilton et al., 2017] samples and aggregates local neighborhoods

but similarly lacks global structural awareness.

None of these use community structure as an organizational principle for attention.

2.2 Knowledge Graph Reasoning

Early KG reasoning systems used rule-based approaches (AMIE [Galarraga et al.,

2013]) or embedding-based methods (TransE [Bordes et al., 2013], RotatE [Sun

et al., 2019]). These methods produce entity embeddings but do not perform

multi-hop traversal in the attention-mechanism sense.

Path-based reasoning approaches (DeepPath [Xiong et al., 2017], MINERVA [Das

et al., 2018]) use reinforcement learning to find paths. These are closer to

our goal but require training data and do not use community structure.

2.3 LLM + KG Hybrid Systems

KG-GPT [Yao et al., 2023], KGPT [Chen et al., 2020], and related systems

connect KGs to LLMs as retrieval stores. The LLM always performs the

reasoning step; the KG is passive.

GraphRAG [Edge et al., 2024] uses community detection (Leiden)

to summarize graph clusters as text, which is then passed to an LLM for

RAG. This is the closest existing work. However:

Communities are summarized as static text chunks, not used as attention heads

The LLM performs all reasoning

Paths are not returned or made interpretable

The system is not graph-agnostic (designed for Microsoft's pipeline)

RAPTOR [Sarthi et al., 2024] builds hierarchical text clusters for RAG but

operates on documents, not graphs, and uses tree structure rather than

community structure.

2.4 Community Detection

The Louvain algorithm [Blondel et al., 2008] optimizes modularity greedily

but can produce internally disconnected communities. Leiden [Traag et al.,

2019] fixes this with a refinement phase promoting internal connectivity.

Label Propagation [Raghavan et al., 2007] is fast and unsupervised but

non-deterministic and produces variable quality.

DSCF (this work) combines LPA and Leiden signals simultaneously at each

node update, using a temperature-annealing schedule. See Section 4.2.

DSCF is non-deterministic (inherits LPA’s shuffle-order sensitivity).

For reproducible results: run 3-5 trials and select the partition with

highest modularity score. For production: seed the RNG and document the seed.

1) The Structural Equivalence

We establish a complete operational mapping between Transformer components

and KG operations. This is not analogy — each mapping is functional.

Transformer Component
CEREBRUM (KG) Equivalent
Notes
Token
Entity or Relation
Atomic unit of information
Vocabulary
Entity type taxonomy
Closed set of possible types
Token embedding
Entity embedding (TransE/RotatE)
Dense vector per entity
Positional encoding
Structural encoding (PR, BW, deg)
Where entity sits globally
Attention head
Community cluster (DSCF)
Specialized relational context
Attention weight
CSA weight formula
Sim + community + edge + distance
Context window
Ego-network radius R
How far to traverse
Layer depth L
BFS hop count
Reasoning step count
Feed-forward sublayer
Entity-type projection
Type-specific transformation
Residual connection
Previous-hop embedding
Prevents information loss
Layer normalization
Embedding normalization
Prevents value explosion
Output projection
Path decoder / ranker
Maps traversal to answer
KV cache
Materialized path store
Reuse traversal across queries

This equivalence has a critical implication: the number of attention heads is

not a hyperparameter in CEREBRUM — it is determined by the graph’s own community

structure. A graph with 12 natural communities has 12 attention heads. A graph

with 200 communities has 200. The architecture adapts to the data.

1) The CEREBRUM Architecture

4.1 Community-Structured Attention (CSA)

CSA computes attention weights for graph traversal that incorporate both local

graph topology and global community structure.

Attention weight formula. For entity u attending to entity v at traversal hop k :

$$a(u, v, k) = \sigma(\alpha \cdot \cos(\mathbf{e}_u, \mathbf{e}_v) + \beta \cdot S_C(u, v) + \gamma \cdot w_{\text{rel}} - \delta \cdot d_{\text{norm}}(u, v)) \quad (1)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid activation, $\cos(\mathbf{e}_u, \mathbf{e}_v) = \frac{\mathbf{e}_u \cdot \mathbf{e}_v}{\|\mathbf{e}_u\| \|\mathbf{e}_v\|}$ is the cosine similarity of entity embeddings, $d_{\text{norm}}(u, v) = d_G(u, v) / \text{diam}(G)$ is the normalized graph distance, and $\phi(k) = \frac{1}{1+k}$ is the hop-depth decay.

Default zero-shot parameters: $\alpha = 0.4$ (embedding similarity), $\beta = 0.4$ (community membership), $\gamma = 0.1$ (edge type), $\delta = 0.05$ (distance penalty), $\varepsilon = 0.05$ (hop decay). All parameters can be learned from (q, a) pairs in supervised settings.

Community membership score. Let $c : V \rightarrow \mathbb{Z}_{\geq 0}$ be the community assignment and $\mathcal{A}$ the set of adjacent community pairs (connected by ≥ 1 cross-community edge):

$$S_C(u, v) = \begin{cases} 1.0 & \text{if } c(u) = c(v) \quad [\text{same attention head}] \\ 0.5 & \text{if } (c(u), c(v)) \in \mathcal{A} \quad [\text{adjacent heads}] \\ e^{-\lambda \cdot d_C(c(u), c(v))} & \text{otherwise} \quad [\text{distance decay, } \lambda = 0.5] \end{cases} \quad (2)$$

where d_C is the shortest path (in hops) between communities in the community-level graph, precomputed once after DSCF converges.

Bridge Bonus $w_{\text{rel}} \in [0, 1]$: an optional per-relation-type weight that explicitly rewards cross-domain edges when the domain structure requires them (default 0.0; recommended 0.4 for inter-type reasoning tasks such as *Movie* $\rightarrow$ *Actor* $\rightarrow$ *Director*).

Why CSA is not a GAT. GATs compute $a(u, v) = f(\mathbf{W}\mathbf{e}_u, \mathbf{W}\mathbf{e}_v)$ from learned weights on adjacent pairs only, with no global context. CSA introduces the term $\beta \cdot S_C(u, v)$ which provides global structural awareness at $O(n \cdot k \cdot C)$ — far cheaper than Transformer self-attention’s $O(n^2)$.

4.2 Dual-Signal Community Fusion (DSCF)

DSCF is the community detection algorithm that produces the attention head

structure. It is a key contribution in its own right.

The core innovation: at each individual node update, both the LPA

majority-vote signal (local topology) and the modularity gain signal (global

structure) are computed. The decision incorporates both simultaneously,

governed by a temperature parameter:

LPA signal (local majority vote):

$$\text{lpa_cid}(v) = \arg \max_c \sum_{u \in \mathcal{N}(v)} \mathbf{1}[c(u) = c], \quad \text{lpa_conf}(v) = \frac{\sum_{u \in \mathcal{N}(v)} \mathbf{1}[c(u) = c]}{\deg(v)} \quad (3)$$

Modularity gain signal (global structure). For each candidate community $\mathcal{C}$ adjacent to v :

$$\Delta Q(v \rightarrow \mathcal{C}) = \frac{k_{v,\mathcal{C}}}{m} - \rho \cdot \frac{k_v \cdot \sum_{u \in \mathcal{C}} k_u}{2m^2} \quad (4)$$

$$\text{mod_cid}(v) = \arg \max_c \Delta Q(v \rightarrow \mathcal{C}), \quad \text{mod_conf}(v) = \min_c (\Delta Q(v \rightarrow \mathcal{C})) \quad (5)$$

where $k_{v,\mathcal{C}}$ is the number of edges from v to $\mathcal{C}$, $k_v = \deg(v)$, m is the total edge count, and ρ is the resolution parameter.

Decision rule (temperature $\tau \in [0.01, 1.0]$):

Condition	Action
$\text{lpa_cid} = \text{mod_cid} \neq c(v)$	MOVE (consensus anchor)
Both signals say STAY	STAY
LPA says MOVE, Mod says STAY	MOVE with probability $\text{lpa_conf} \cdot \tau$
Mod says MOVE, LPA says STAY	MOVE with probability $\text{mod_conf} \cdot (2 - \tau)$
Both say MOVE to different targets	Move to lpa_cid with weight $\text{lpa_conf} \cdot \tau$; to mod_cid with weight $\text{mod_conf} \cdot (2 - \tau)$

Temperature schedule (anneals from local-dominant to global-dominant):

$$\tau_{t+1} = \max(\tau_t \cdot \alpha_{\text{cool}}, \tau_{\min}), \quad \alpha_{\text{cool}} = 0.92, \quad \tau_{\min} = 0.01 \quad (6)$$

Post-convergence connectivity check. Any community $\mathcal{C}$ whose induced subgraph $G[\mathcal{C}]$ is disconnected is split into its connected components:

$$\text{If } G[\mathcal{C}] \text{ is disconnected: } \mathcal{C} \rightarrow \mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_r \quad (7)$$

Why DSCF communities are the right attention heads:

In a trained Transformer:

Some heads specialize on local structure (syntactic patterns, adjacent tokens)

Other heads specialize on long-range structure (coreference, semantic themes)

The combination allows both local and global reasoning simultaneously

DSCF communities exhibit exactly this dual character. The LPA component supports

communities are locally coherent (nodes in the same community are topologically

close). The modularity component supports communities are globally significant

(they represent structurally distinct regions of the graph). A node that is in

a DSCF community is there because both local and global signals agreed.

This dual property has not previously been used as a basis for attention mechanisms in any published graph learning system.

Comparison to Leiden-only communities:

Leiden optimizes purely for modularity. On sparse regions of a graph, Leiden

may split locally coherent neighborhoods across multiple communities because

the global modularity gain favors a different partition. DSCF resists this —

the LPA component holds locally coherent groups together even when modularity would split them.

Comparison to LPA-only communities:

LPA can merge structurally distinct regions if they happen to be locally

connected (the "resolution limit" problem). DSCF resists this — the

modularity signal penalizes over-merging.

1) The Forward Pass: Graph Reasoning

5.1 Algorithm

INPUT: Query Q (text string or entity list)

Graph G (any backend)

Community assignments $\mathcal{C}$ (from DSCF)

Entity embeddings E (from any KGE method)

Hop depth L , beam width B , top- K

OUTPUT: Ranked list of reasoning paths $P = [(\text{path}, \text{score}, \text{explanation})]$

Step 1 — Entity Grounding. Extract seed entities $S = \{e_1, \dots, e_n\}$ from query Q (via NER or n-gram fuzzy match).

Step 2 — Structural Encoding. For each seed entity e_i with positional vector $\mathbf{p}_i = [\text{PR}(e_i), \text{BW}(e_i), \text{deg}(e_i)]^\top$:

$$\mathbf{h}_i^{(0)} = \text{LayerNorm}(\mathbf{e}_i + \mathbf{W}_{\text{pos}} \cdot \mathbf{p}_i) \quad (8)$$

Step 3 — Attention Traversal ($k = 1, \dots, L$). Initialize beam $\mathcal{B}^{(0)} = \{(e_i, \mathbf{h}_i^{(0)}, 1.0)\}$. At each hop:

Embedding aggregation when extending to neighbor v with attention weight $w = a(u, v, k)$:

$$\mathbf{h}^{(k)} = \text{LayerNorm}(\mathbf{h}^{(k-1)} + \text{ReLU}(w \cdot \mathbf{e}_v + \mathbf{h}^{(k-1)})) \quad (9)$$

($\mathbf{W}_k = \mathbf{I}$ in zero-shot deployment; learned per-hop in supervised settings)

Path score update:

$$s(\mathcal{P} \oplus v) = s(\mathcal{P}) \cdot a(u, v, k) \cdot \gamma_{\mathcal{C}}(\text{cseq}(\mathcal{P}) \cup \{c(v)\}) \quad (10)$$

Beam pruning:

$$\mathcal{B}^{(k)} = \text{top}_B(\text{candidates}^{(k)}, \text{key} = s) \quad (11)$$

STEP 4 — Path Scoring

Final score for path $P = (e \rightarrow r \rightarrow e \rightarrow r \rightarrow \dots \rightarrow e)$:

$\text{score}(P) = \text{attention_weights}$

$\times \text{community_coherence}(P)$

$\times \text{semantic_alignment}(\mathbf{h}_{\text{final}}, \text{query_embedding})$

STEP 5 — Output

Return top-K paths ranked by score

Each path includes:

- Ordered entity/relation sequence
- Score breakdown (attention, community, semantic)
- Community sequence (which "heads" were traversed)
- Natural language explanation template

5.2 Community Coherence

The `community_coherence` term rewards paths that traverse communities in a

principled way:

`community_coherence(path) =`

$(\text{intra-community steps} \times 1.0 + \text{cross-community steps} \times 0.5)$

`/ total steps`

A path that stays within one community scores 1.0 (tight local reasoning).

A path that makes one community transition scores 0.75 (one conceptual leap).

This prevents paths that jump incoherently across unrelated domains.

Step 4 — Path Scoring. Final score for path $P = (v_0, r_1, v_1, \dots, r_L, v_L)$:

$$\text{score}(P) = \left(\prod_{k=1}^L a(u_k, v_k, k) \right) \cdot \gamma_C(P) \cdot \cos(\mathbf{h}_L, \mathbf{q}) \quad (12)$$

Community coherence (Step 3 and 4 shared function):

$$\gamma_C(P) = \frac{1}{L} \sum_{k=1}^L \begin{cases} 1.0 & c(v_k) = c(v_{k-1}) \\ 0.5 & c(v_k) \neq c(v_{k-1}) \end{cases} \quad (13)$$

A fully intra-community path scores $\gamma_C = 1.0$. One community transition: $\gamma_C \approx 0.75$. Incoherent zigzags compound the penalty exponentially across hops.

5.3 Interpretability

The output of CEREBRUM is always a path through the KG. This is fundamentally

different from LLM reasoning in two ways:

Verifiable: every step is an explicit (entity, relation, entity) triple

that exists in the graph. The system cannot hallucinate a connection that isn't there.

Auditable: the community sequence tells you which conceptual domains

were traversed. A path that crosses from community "Clinical Trials" to

community "Drug Mechanisms" to community "Side Effects" is immediately understandable.

This property makes CEREBRUM specifically valuable in high-stakes domains

(medical, legal, financial) where hallucination is unacceptable.

1) Implementation Architecture**6.1 Design Principles**

Framework agnostic: CEREBRUM must work with any graph database, any

embedding method, and any LLM (or no LLM). No vendor lock-in.

No training required by default: The zero-shot configuration uses fixed

parameters ($\alpha, \beta, \gamma, \delta, \epsilon$) and any entity embedding. Communities are

computed unsupervised via DSCF.

Progressive enhancement: Users can improve performance by providing

training pairs (for learning parameters) or domain-specific edge type weights

without changing the core architecture.

Minimal dependencies:

Core: networkx, numpy, leidenalg (for DSCF), scipy

Adapters: optional graph DB drivers

Embeddings: optional sentence-transformers or pykeen (for TransE/RotatE)

API: optional FastAPI

6.2 Repository Structure

parallax/

core/

__init__.py

graph_adapter.py # Abstract base class for graph backends

embedding_engine.py # Entity embedding interface

community_engine.py # Community detection (DSCF + others)

attention_engine.py # Community-Structured Attention

structural_encoder.py # Graph positional encoding

reasoning/

traversal.py # Beam-search attention traversal

path_scorer.py # Multi-signal path ranking

answer_extractor.py # Extract top-K answers

adapters/

networkx_adapter.py # In-memory (default, no external deps)

neo4j_adapter.py # Neo4j via bolt

rdf_adapter.py # SPARQL endpoint (Wikidata, DBpedia)

csv_adapter.py # Bootstrap from edge-list CSV

llm_bridge/

context_formatter.py # Optional: format paths as LLM

context

api/

server.py # FastAPI REST server

schemas.py # Pydantic models

cli/

parallax.py # Command-line interface

tests/

test_dscf.py

test_csa.py

test_traversal.py

fixtures/

toy_graph.csv # Small graph for unit tests

benchmarks/

webqsp_eval.py # WebQSP KG Q&A benchmark

metaqa_eval.py # MetaQA multi-hop benchmark

baseline_comparison.py # CSA vs GAT vs GraphRAG

examples/

Validation_Walkthrough.ipynb # Interactive visual proof

wikidata_quickstart.py
 neo4j_quickstart.py
 csv_quickstart.py
 pyproject.toml
 README.md
 PAPER.md # Living research document
 6.3 The Abstract Graph Adapter

The adapter pattern is what makes CEREBRUM truly agnostic:

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import List, Optional
@dataclass
class Entity:
    id: str
    label: str
    type: str
    properties: dict
@dataclass
class Edge:
    source_id: str
    target_id: str
    relation_type: str
    weight: float = 1.0
class GraphAdapter(ABC):
    @abstractmethod
    def get_entity(self, entity_id: str) -> Optional[Entity]: ...
    @abstractmethod
    def get_neighbors(self, entity_id: str,
        edge_types: List[str] = None,
        max_neighbors: int = 50) -> List[Edge]: ...
    @abstractmethod
    def find_entities(self, query: str, top_k: int = 10) -> List[Entity]: ...
    @abstractmethod
    def to_networkx(self) -> "nx.Graph": ... # for community
detection
```

Any graph system that implements these four methods works with the full

CEREBRUM stack. NetworkX, Neo4j, RDF SPARQL, property graph CSV — all handled by their respective adapter.

6.4 What Comes From Home Assistant

The following components are directly portable from Home Assistant with minor

generalization:

dscf_communities() — pure Python, depends only on networkx

leiden_communities() / lpa_communities() / hybrid_communities()

Neo4j connection patterns and Cypher templates

The community broadcast WebSocket pattern (for live applications)

No other Home Assistant-specific dependencies are carried over.

6.5 Phased Build Plan

Phase 0 — Theory (complete)

Formalize CSA and DSCF; write white paper

Prototype DSCF in Python (done: lives in Home Assistant knowledge_service)

Validate DSCF produces stable communities on Home Assistant's Neo4j graph

Phase 1 — Core Engine

core/graph_adapter.py — abstract base + NetworkX adapter

core/community_engine.py — DSCF, Leiden, LPA, hybrid (ported from Home Assistant)

core/embedding_engine.py — SentenceEngine (zero-training default)

core/attention_engine.py — CSA weight formula

core/structural_encoder.py — PageRank, betweenness, degree encoding

Unit tests on fixtures/toy_graph.csv for all of the above

Phase 2 — Reasoning Engine

reasoning/traversal.py — beam-search traversal with CSA weights

reasoning/path_scorer.py — multi-signal scoring + community coherence

reasoning/answer_extractor.py — top-K ranked answers

Integration test: end-to-end query on toy graph produces grounded paths

Phase 3 — Adapters + API

adapters/neo4j_adapter.py (port Home Assistant patterns)

adapters/rdf_adapter.py (SPARQL, for Wikidata/DBpedia)

adapters/csv_adapter.py (bootstrap from edge-list)

api/server.py — FastAPI REST: /query, /communities, /health

cli/parallax.py — command-line interface

Phase 4 — LLM Bridge + Benchmarks

llm_bridge/context_formatter.py — format paths as LLM prompts

benchmarks/webqsp_eval.py and metaqa_eval.py

Ablation study: DSCF vs Leiden vs LPA as attention heads

Baseline comparisons: BFS, GAT, GraphRAG, vanilla RAG

Phase 5 — Release

Write formal paper from white paper foundation

Submit to venue (e.g., EMNLP, ICLR, NeurIPS Graphs Track)

Open-source repository under chosen license

6.6 Computational Complexity

DSCF community detection: $O(E \cdot I)$ where E = edges, I = iterations

(typically $I < 50$ before convergence). Comparable to Louvain.

Community graph construction: $O(E)$ post-DSCF; pre-computed once.

CSA weight per edge: $O(d)$ where d = embedding dimension. Precompute

structural encodings once; community lookups are $O(1)$ with a hash table.

Beam traversal (one query): $O(B \cdot L \cdot k \cdot d)$ where:

B = beam width (default 10)

L = hop depth (default 3)

k = average degree

d = embedding dimension

For a graph with $k=20$, $L=3$, $B=10$, $d=384$: 230,000 floating-point operations

per query — milliseconds on CPU, no GPU required.

Comparison to Transformer: Full attention is $O(n^2 \cdot d)$ per layer.

CEREBRUM traversal is $O(B \cdot L \cdot k \cdot d)$ — independent of graph size n .

This makes CEREBRUM sublinear in graph size for fixed-width beam search.

6.7 Known Failure Modes

Dense hub nodes: Nodes with very high degree ($k \gg k$) cause beam

explosion at that hop. Mitigation: cap `max_neighbors` per node (default 50).

Homogeneous graphs: If all nodes share the same community, CSA degenerates

to pure embedding similarity (`community_score` terms cancel). This occurs in

highly regular graphs (grids, complete bipartite). Mitigation: check community

count post-DSCF; if count < 3 , fall back to BFS with embedding similarity only.

Disconnected graphs: Multiple connected components each get their own

communities; cross-component traversal is impossible by definition. Queries

spanning components will return no paths. Mitigation: surface component

boundaries to callers; allow separate per-component queries.

Sparse embedding coverage: Entities with generic or missing labels get

near-random sentence embeddings. The α term (embedding similarity) becomes

noise; DSCF community structure (β term) carries the full weight. Still

functions; interpretability of similarity scores is reduced.

Adversarial community injection: A malicious actor who can insert edges

into the graph can manipulate community structure and thus attention weights.

Relevant for applications where the KG is user-writable. Mitigation: sign trusted edges; treat unsigned-edge-induced communities with lower β weight.

6.8 Horizontal Scalability

BLUF: CEREBRUM query serving scales horizontally without architectural limit. Queries are stateless reads from a sharded graph database; throughput grows linearly with workers. LLM inference cannot do this.

CEREBRUM's scalability is bounded by the scalability of graph databases — a solved problem at web scale — not by neural network inference constraints.

Within a single query: at each beam hop, all $B \times k$ candidate edge scores are computed independently. The only synchronization point is the top-B selection (200 items). GPU threads handle all scoring simultaneously with no data dependency between candidates.

Across concurrent queries: traversal workers are stateless during inference (graph and embeddings are read-only). Throughput scales linearly with worker count; zero inter-query synchronization is required.

Across machines: CSA's community preference (score = 1.0 intra-community vs. exponential decay inter-community) makes communities the natural shard key. Most queries never cross a shard boundary. Cross-shard traversal is a sparse network call, implemented in the FederatedAdapter (Phase 6–9). The holographic index identifies remote entity ownership without loading remote data.

Amdahl's Law comparison:

	LLM inference	CEREBRUM query
Sequential fraction	100% (layer N waits for layer N-1)	0.01% (top-B select per hop)
Cross-machine sync	All-reduce at every layer	Zero within a shard
GPU speedup ceiling	8–16 per inference call	Linear with workers
Knowledge touched per call	100% of parameters	0.001% of graph

Community detection (DSCF) is the only component requiring a global graph view. It runs offline and periodically — never on the query critical path. Queries run against the last known partition while the next detection pass executes in the background.

The one-line summary: a KG query touches 0.001% of the graph per call using embarrassingly parallel operations. An LLM touches 100% of its parameters every time, sequentially. That gap defines the scalability story.

1) Experimental Results

7.0 Experimental Environment

All benchmarks were executed on the following hardware and software configuration to support reproducibility:

- CPU: AMD Ryzen 9 9950X3D 16-Core Processor (32 Logical Processors)
- RAM: 64 GB DDR5
- OS: Windows 11 Pro (Build 10.0.26220)
- Python: 3.14.0
- Graph Backends: NetworkX 3.4.2, igraph 0.11.6
- Embeddings: RandomEngine (64-dim) for structural validation; SentenceEngine (384-dim) for semantic tasks.

7.1 MetaQA: The Baseline Lower Bound

MetaQA evaluation revealed a Structural Mismatch (EF-004). Because MetaQA answer paths always cross entity-type boundaries (Movie $\rightarrow$ Actor), and community detection naturally separates these types, the default CSA formula (favoring intra-community edges) penalized the correct paths. Outcome: BFS outperformed CSA variants on Hits@1. Significance: Established the "lower bound" of performance on topologies where community signal is anti-informative.

7.2 The Bridge Bonus Innovation (EF-005)

To solve the "Type Alignment Trap" identified in MetaQA and Hetionet, we introduced the Metaedge Bridge Bonus (`w_rel` in the CSA formula). By assigning a positive bonus (e.g., 0.4) to inter-type metaedges like treats or associates, we offset the cross-community penalty while retaining structural guidance.

7.3 Hetionet: Biomedical Reasoning at Scale

On a 500,000-edge subset of Hetionet, CEREBRUM with LPA attention heads and the Bridge Bonus significantly outperformed the BFS baseline.

- `disease_associates_gene`: LPA+CSA H@1 0.6560 vs BFS 0.4320 (+51.8%)
- `gene_participates_pathway`: LPA+CSA H@1 0.2600 vs BFS 0.0950 (+173.6%)

7.4 WebQSP: Real-world Entity Lookup

On the WebQSP benchmark (FB15k-237), CEREBRUM demonstrated superior recall and ranking quality.

- CEREBRUM (LPA+CSA): Hits@10 0.3360, MRR 0.1203
- BFS Baseline: Hits@10 0.3000, MRR 0.1081

7.5 Key Findings

- 1) **Recall Advantage**: CSA variants consistently achieve higher recall (Hits@10) than BFS, validating the system’s ability to steer the beam toward correct graph regions.
- 2) **Signal Duality**: DSCF provides finer-grained precision, while LPA provides coarser, more robust recall.
- 3) **Zero-Shot Viability**: All results were achieved using random embeddings and manual weights, proving CEREBRUM works without any training data.
- 4) **The DSCF-as-Attention-Head Hypothesis**

The central theoretical claim of CEREBRUM is that DSCF communities are better attention heads than Leiden-only or LPA-only communities. We state this as a falsifiable hypothesis:

H1 (DSCF Attention Hypothesis): For multi-hop reasoning tasks on KGs, CEREBRUM with DSCF attention heads achieves higher answer accuracy than CEREBRUM with Leiden-only or LPA-only attention heads.

H2 (CSA vs GAT Hypothesis): CSA-guided traversal achieves higher accuracy on multi-hop questions than GAT-based traversal on the same graph and same entity embeddings.

H3 (Interpretability Hypothesis): CEREBRUM paths receive higher human coherence ratings than equivalent LLM-generated reasoning chains on the same questions, because every step is a grounded graph edge.

H3 Evaluation Protocol: Present matched pairs — one CEREBRUM path and one LLM reasoning chain — to $N \geq 30$ annotators blind to their source. Ask:

"Which reasoning chain is more coherent and trustworthy? (A / B / Equal)".

Primary metric: proportion preferring CEREBRUM path. Secondary: Cohen’s kappa for inter-annotator agreement (target $\kappa > 0.6$).

These hypotheses are testable on standard benchmarks (WebQSP, MetaQA-3hop) and define the empirical work for Phase 2.

0.2 9. Benchmarks and Results

0.2.1 9.1 Datasets

Dataset	Task	Hops	Size
WebQSP	Single + multi-hop QA	1-2	4,737 questions
MetaQA-2hop	Multi-hop QA	2	118,980 questions
MetaQA-3hop	Multi-hop QA	3	114,196 questions
FB15k-237	Link prediction	-	310,116 triples
Toy graph (internal)	Unit testing	1-4	200 nodes

0.2.2 9.2 Baselines

System	Type	Notes
BFS (no attention)	Graph traversal	Traversal without CSA weighting
GAT	Graph neural network	2-layer, trained
GraphRAG	LLM-based	Community summaries → GPT-4
RAG (vanilla)	LLM-based	FAISS retrieval → GPT-4
CEREBRUM (TSC)	Graph attention	Ours, Triple-Signal Consensus heads
CEREBRUM (LPA)	Graph attention	Ablation: LPA-only heads

0.2.3 9.3 Metrics

- **Hits@1, Hits@3, Hits@10**: answer in top-K paths
- **Mean Reciprocal Rank (MRR)**: ranked answer quality
- **Path coherence** (human eval): are the reasoning paths understandable?
- **Grounding rate**: what fraction of returned paths are fully grounded

(all edges verified in the KG)?

0.2.4 9.4 Phase 4 Results (Ablation Study)

A rigorous ablation study on MetaQA (Run 019) yielded the following key engineering findings:

- 1) **Structural Mismatch (EF-004)**: Breadth-First Search (BFS) consistently outperforms CSA variants on MetaQA. This confirms that MetaQA’s question structure (cross-type entity lookup) is penalized by community-based attention, which favors intra-community coherence. This is a dataset-specific characteristic, not an algorithmic defect.
- 2) **TSC Stability**: The Triple-Signal Consensus (TSC) engine demonstrated superior stability in community detection compared to earlier DSCF iterations, producing consistent partition counts across runs.
- 3) **The Mesoscale Gap**: TSC produces fine-grained communities (14k on MetaQA) compared to LPA (1.6k). This granularity provides high precision for local queries but necessitates the "Metaedge Bridge Bonus" or Federated Reasoning strategies to bridge large topological distances in multi-hop tasks.

0.3 10. Open Research Questions

9.1 Embedding Strategy

Two options exist:

Option A — Pre-trained structural embeddings (TransE/RotatE): trained on the graph structure itself. More precise but requires a training step. Suitable for static KGs or when training compute is available.

Option B — On-the-fly label embeddings (sentence-transformers): encode entity labels and descriptions using a pre-trained language model. No graph-specific training needed. More agnostic. Less precise for entities with ambiguous labels.

Recommended default: Option B for zero-shot deployment; Option A when the graph has been stable and

training is feasible. CEREBRUM should support both interchangeably via the Embedding Engine interface.

9.2 Adaptive Community Granularity

The DSCF resolution parameter controls how many communities are formed. Too few communities = coarse attention heads that miss structure. Too many = noisy heads that don't generalize.

Proposed adaptive rule: target $K \approx N$ communities, where N = node count.

This is consistent with theoretical results on optimal modularity resolution and supports attention head count scales sensibly with graph size.

For Home Assistant's KG ($N \approx 5,000$): target 70 communities.

For Wikidata subset ($N \approx 100,000$): target 316 communities.

9.3 Soft vs Hard Community Membership

DSCF produces hard assignments (each node belongs to exactly one community).

Real-world entities often span multiple communities — a person can be both a scientist and a politician.

Extension: weight-based soft membership, where each node has a probability distribution over communities. The community score function becomes a dot product of membership vectors. This would require modifying DSCF to track confidence scores at convergence.

9.4 Learnable Parameters

In zero-shot mode, $\alpha, \beta, \gamma, \delta, \epsilon$ are fixed. For supervised settings, they can be learned from (query, ground-truth-answer) pairs via gradient descent on a path-ranking loss. This is an optional enhancement that does not affect the core architecture.

9.5 Temporal Knowledge Graphs

Time-stamped KGs (events, evolving relationships) introduce a temporal dimension. The positional encoding would need to incorporate temporal distance alongside graph-structural distance. Left for future work.

9.6 Triple-Signal Consensus (TSC): Closing the Mesoscale Gap

A significant extension for Phase 2 research is the transition from the dual-signal DSCF to a Triple-Signal Consensus (TSC) framework. This addresses the "Mesoscale Gap"—the structural region between immediate local topology (LPA) and global modularity (Leiden).

By introducing a third methodology, such as Infomap (Map Equation), we can weed out "structural hallucinations"—paths that exist in the graph but lack conceptual coherence. While modularity captures static edge density, Infomap captures the flow of information (random walks), identifying bottlenecks and sub-clusters that static methods often miss.

In this framework, a node move or a traversal edge must pass a Consensus Filter:

- 1) LPA (Local): Immediate neighbor recognition.
- 2) Modularity (Global): Global architecture optimization.
- 3) Infomap (Mid-Level): Natural information flow transit.

The resulting decision logic for community fusion evolves into a tri-signal fused probability: $P(\text{move}) = f(\text{LPA} \cdot \tau_{\text{local}}, \text{Mod} \cdot \tau_{\text{global}}, \text{Infomap} \cdot \tau_{\text{mid}})$

This "mid-level voting" helps confirm that only the most structurally robust reasoning chains survive the beam-search pruning process.

1) Broader Impact and Applications

10.1 Domain Applications

Biomedical: Drug-gene-disease-pathway graphs. Multi-hop reasoning for drug repurposing ("Drug X inhibits enzyme Y which is overexpressed in disease Z"). Grounded inference is critical — no LLM should hallucinate drug interactions.

Legal: Case law citation and statutory reference networks. Multi-hop precedent tracing. Every step in a legal argument must be citable; CEREBRUM's grounded paths match this requirement exactly.

Cybersecurity: Attack graphs, CVE dependency networks. "What path leads from this exposed service to root access?" — a life-safety question that benefits from verified, traceable reasoning chains.

Software engineering: Code dependency and call graphs. Impact analysis: "What does changing function X affect?" traversed as a multi-hop attention path with community context (same module = high attention).

Finance: Entity relationship graphs for regulatory compliance. Traceable reasoning chains for auditors: "Why did this transaction trigger a flag?"

10.2 The LLM Bridge

CEREBRUM is designed to augment LLMs, not replace them. The `llm_bridge` module formats traversal output as structured context:

You are reasoning about: [query]

The knowledge graph traversal found these paths:

Path 1 (score: 0.94): Marie Curie [COMMUNITY: Scientific Discoveries] → [discovered] → Polonium [COMMUNITY: Scientific Discoveries] → [exhibits] → Radioactivity [COMMUNITY: Physics Phenomena]

Please summarize what this tells us about [query] in natural language.

This gives any LLM a grounded, structured context that minimizes the risk of hallucination because the facts are provided explicitly. The LLM's role is purely natural language generation, not reasoning.

10.3 The Agnosticism Property

CEREBRUM is agnostic across five dimensions:

Graph database: implement GraphAdapter for any system
Embedding method: implement embeddingEngine for any model LLM: any model or none — CEREBRUM works without one
Domain: the algorithm is domain-blind; community structure emerges from the graph's own topology
Query language: entities can be identified from text, IDs, or direct lookup — the entry point is flexible

10.4 National Security and Multi-Source Applications

>**Note:** The following applications are **potential and unverified**. They represent the theoretical utility of the CEREBRUM architecture for national security but require empirical validation on representative datasets.

CEREBRUM's **Glass-Box** architecture and **Zero-Hallucination** capability make it a primary candidate for complex reasoning tasks.

Potential Application: Signal Analysis: Mapping emitter-relationship networks. By treating signal emitters

as nodes and temporal or spatial co-occurrence as edges, CEREBRUM could identify command-and-control hierarchies through TSC community detection and trace information flow through CSA traversal.

Potential Application: Open Source Analysis: Tracking information operations. Tracing the propagation of narratives across social graphs with verifiable provenance. Identifying "source communities" of disinformation campaigns without Black-Box speculation.

Potential Application: Social Network Analysis: Social Network Analysis (SNA). Identifying "gatekeeper" entities between clandestine communities using betweenness centrality. Validating informant claims by checking for path consistency across a grounded knowledge base.

Potential Application: Logistical Analysis: Logistical reasoning. Connecting facilities, equipment, and transit routes extracted from imagery into a reasoning graph. Tracing movement patterns to identify supply chain anomalies or facility purpose.

Potential Application: Signature Correlation: Signature correlation. Reasoning over networks of technical signatures (acoustic, seismic, chemical). Identifying anomalous clusters that correspond to specific industrial or technological processes.

1) Production Hardening (Phase 10)

Phase 10 hardened the research prototype into a production-grade service deployable in real environments. Three components were introduced: authentication, resource governance, and asynchronous streaming traversal.

11.1 JWT Authentication

All API endpoints are protected by JSON Web Token (JWT) bearer authentication. Tokens carry an expiry claim and are validated on every request by the FastAPI dependency injection layer. Unauthenticated requests receive HTTP 401; expired tokens receive HTTP 403. Token issuance is intentionally decoupled from CEREBRUM itself, enabling integration with any OAuth2 or OIDC identity provider.

11.2 Resource Governor

Unbounded queries over large graphs risk exhausting system memory or monopolizing CPU. The ResourceGovernor enforces per-request ceilings:

Parameter	Default	Description
max_nodes	50,000	Node count above which traversal is refused
max_edges	200,000	Edge count ceiling
max_beam_size	50	Beam width cap
max_hops	6	Depth ceiling per query
query_timeout_s	30.0	Wall-clock timeout in seconds
max_concurrent	10	Concurrent query limit

Queries that exceed any ceiling receive HTTP 429 with a structured JSON error explaining which limit was breached.

11.3 Asynchronous Beam Traversal

The `AsyncBeamTraversal` class wraps `BeamTraversal` in Python's `asyncio` event loop and adds a `/query/stream` SSE endpoint. Instead of blocking until the full beam completes, the server emits one Server-Sent Event per beam step:

```
event: step
data: {"hop": 1, "candidates": [...], "scores": [...]}

event: result
data: {"paths": [...], "top_score": 0.94, "hops": 2}
```

This allows front-ends and downstream agents to render partial results as the traversal progresses — important for user-facing applications where a 3-hop traversal over a large graph may take several seconds.

The `/query/stream` endpoint accepts the same request schema as `/query` and is protected by the same JWT authentication. The `ResourceGovernor` timeout applies per stream: if traversal does not complete within `query_timeout_s` seconds, the stream emits a final event: `timeout` and closes.

1) Real-Time Streaming (Phase 11)

Phase 11 extended CEREBRUM from static knowledge graph reasoning to live, streaming data. The motivation is direct: many of CEREBRUM's highest-value applications — industrial sensors, network telemetry, signal analysis, video feeds — produce continuous data that must be reasoned over without batch processing delays.

12.1 The Streaming Architecture

The streaming subsystem has four interacting layers:

- 1) **Sources** — pluggable data ingestion adapters (file tail, HTTP polling, WebSocket, MQTT, Python callbacks)
- 2) **Discretizer** — converts continuous signal values into discrete graph nodes and typed edges
- 3) **SlidingWindowBuffer** — maintains a time-bounded, count-bounded live edge set with reference counting
- 4) **StreamAdapter** — a thread-safe, live-mutating `NetworkXAdapter` that integrates all layers

12.2 Sliding Window Buffer

The buffer maintains the set of currently-active edges using two eviction policies applied together:

$$\mathcal{W}(t) = \{e \in \mathcal{E} \mid t - t_e \leq \Delta t\} \cap \{|\mathcal{E}| \leq N_{max}\} \quad (14)$$

where Δt is the time window in seconds, t_e is the most recent timestamp for edge e , and N_{max} is the maximum edge count. When both policies would evict different edges, the buffer evicts the oldest-first until both constraints are satisfied.

Reference counting tracks how many live `StreamEvent` objects reference each edge key (src, rel, tgt). An edge is only removed from the graph when its reference count drops to zero, preventing premature eviction of frequently-repeated signals.

12.3 Signal Discretization

Continuous sensor values are converted to discrete graph edges through one of five discretizer classes:

ThresholdDiscretizer maps a scalar reading to a categorical state node with hysteresis:

$$\text{state}(x) = \begin{cases} \text{SPIKE} & x > \mu + 3\sigma \\ \text{HIGH} & x > \mu + \sigma \\ \text{LOW} & x < \mu - \sigma \\ \text{NORMAL} & \text{otherwise} \end{cases} \quad (15)$$

where μ is a configurable nominal value and σ is the threshold scale. The hysteresis margin h requires the signal to exceed $\theta \pm h$ before transitioning, preventing rapid edge oscillation at threshold boundaries.

BinningDiscretizer quantizes x into N equal-width bins over $[\min, \max]$:

$$\text{bin}(x) = \left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot N \right\rfloor \quad (16)$$

Each bin maps to a node label (e.g., `sensor_A_bin_3`) and emits a `READS` edge from the sensor node to the bin node.

ObjectDetectionDiscretizer converts a detection event (label, confidence, bounding box) into `DETECTS` edges and `CO_OCCURS_WITH` edges between co-detected objects within the same frame.

TemporalSequenceDiscretizer emits `PRECEDES` edges between consecutive events, building an ordered chain of observations:

$$v_1 \xrightarrow{\text{PRECEDES}} v_2 \xrightarrow{\text{PRECEDES}} v_3 \xrightarrow{\text{PRECEDES}} \dots \quad (17)$$

CoActivationDiscretizer emits `CO_ACTIVATES` edges between sensor pairs that fire within a configurable time window δ_t of each other and have co-activated at least $n_{\min}$ times:

$$\text{emit}(A, B) \iff |t_A - t_B| \leq \delta_t \wedge \text{count}(A, B) \geq n_{\min} \quad (18)$$

12.4 Incremental Community Updates

Running full DSCF after every ingested event would be prohibitively expensive. `IncrementalCommunityUpdater` maintains a set of *affected nodes* $\mathcal{A}$ — nodes that gained or lost edges since the last community computation — and re-runs DSCF only over the ego-network of radius r around those nodes:

$$G_{\text{local}} = \{v \mid d_G(v, u) \leq r, u \in \mathcal{A}\} \quad (19)$$

where d_G is shortest-path distance. The default radius is $r = 2$. Updates are triggered when $|\mathcal{A}| \geq n_{\min}$ (default 10), batching small mutations to amortize overhead. The sub-graph is converted to undirected before DSCF runs (since directed sensor graphs use `DiGraph` internally but DSCF requires an undirected view for connected-components computation).

Community assignments outside G_{local} are preserved from the previous full run, producing a hybrid partition that is locally fresh and globally stable.

12.5 Streaming API Endpoints

Endpoint	Method	Description
<code>/stream/ingest</code>	POST	Ingest a batch of <code>StreamEvent</code> objects into the live graph
<code>/stream/status</code>	GET	Live stats: nodes, edges, communities, events/s, buffer utilization
<code>/stream/events</code>	GET (SSE)	Subscribe to graph mutation events in real time

The `/stream/ingest` endpoint accepts:

```
{
  "events": [
    {
      "source": "sensor_A",
      "relation": "READS",
      "target": "temperature_HIGH",
      "timestamp": 1711051234.5,
      "ttl": 60.0,
      "metadata": {"unit": "celsius", "value": 87.3}
    }
  ]
}
```

The `/stream/events` SSE stream emits one JSON line per graph mutation (edge add, edge evict, community update), enabling downstream subscribers to maintain a synchronized view of the live graph.

12.6 Application: Sensor Co-Activation Reasoning

Consider a factory with 50 sensors feeding CEREBRUM via MQTT. The `CoActivationDiscretizer` builds a graph of sensors that reliably activate together. Within minutes of startup, CEREBRUM has formed communities of co-activating sensors — physically meaningful clusters corresponding to cooling loops, power rails, and conveyor stages — without any domain configuration.

A query of the form "which sensors are structurally related to `Sensor_A`?" returns a verified path:

```
>Sensor_A  $\xrightarrow{\text{CO\_ACTIVATES}}$  Sensor_B  $\xrightarrow{\text{CO\_ACTIVATES}}$  Sensor_C
Sensor_C  $\xrightarrow{\text{MONITORS}}$  Cooling
LOOP
2
```

Every edge in this path is a real co-activation event captured in the sliding window. No fact was inferred from a model; every fact was observed.

12.7 Bridge Twin Nodes (Phase 12)

The exponential community distance penalty in the CSA formula correctly discounts speculative cross-domain hops. However, when a specific crossing is used repeatedly — not speculatively but because it is the structurally correct reasoning step — the persistent penalty is a performance cost rather than a correctness signal.

Bridge Twin Formation. When a cross-community traversal of node v into community $\mathcal{C}_d$ occurs $\geq n_{\min}$ times and the node's embedding is semantically close to the destination community centroid:

$$\text{fit}(v, \mathcal{C}_d) = \cos\left(\mathbf{e}_v, \frac{1}{|\mathcal{C}_d|} \sum_{u \in \mathcal{C}_d} \mathbf{e}_u\right) \geq \theta_{\text{bridge}} \quad (20)$$

a bridge twin node v' is materialised with:

- $\mathbf{e}_{v'} = \mathbf{e}_v$ (identical embedding at birth)
- $c(v') = C_d$ (assigned to destination community)
- Bidirectional `BRIDGE_TWIN` edges: $v \leftrightarrow v'$ with $w_{rel} = 1.0$

The twin can subsequently accumulate community-specific edges, diverging from the original to represent the same underlying concept through the lens of its adopted community.

CSA short-circuit for bridge edges. A `BRIDGE_TWIN` edge has $\cos(\mathbf{e}_v, \mathbf{e}_{v'}) = 1.0$ and $S_C(v, v') = 1.0$ by construction. The formula reduces to:

$$a(v, v', k) = \sigma(\alpha + \beta + \gamma + \varepsilon\phi(k)) = \sigma(0.925 + \varepsilon\phi(k)) \approx 0.716 \quad (21)$$

This is implemented as a direct short-circuit in `CSAEngine.compute_weight()`, bypassing the general formula for efficiency and numerical stability.

LTD-analog pruning. Bridge twins idle for $\geq \tau_{prune}$ days are removed from both the engine registry and the adapter graph — exactly as synapses that fall out of use weaken and prune under long-term depression.

Biological correspondence. Bridge twins are the algorithmic equivalent of thalamic relay nuclei: faithful copies of source signals re-expressed in an intermediate structural location, completing bidirectional circuits between otherwise-distant brain regions. The lateral geniculate nucleus holds the same retinotopic map as the retina but lives inside the thalamus, positioned close to the visual cortex it serves. Formation requires repeated use (n_{min} , like the stimulus repetition threshold for LTP), and pruning occurs with disuse (τ_{prune} , like LTD).

12.8 Spike-Timing Dependent Plasticity (Phase 13)

The `CoActivationDiscretizer` (Phase 11) detects *symmetric* correlations: sensors A and B that co-activate are linked with equal weight regardless of which fired first. In real causal systems this symmetry is incorrect — causation has a direction and a time ordering.

Biological STDP resolves this: if neuron A fires **before** neuron B within a short window, the synapse $A \rightarrow B$ is potentiated (LTP). If A fires **after** B, $A \rightarrow B$ is depressed (LTD). The magnitude depends exponentially on the time difference $\Delta t = t_{post} - t_{pre}$:

$$\Delta w(A \rightarrow B) = \begin{cases} A_+ \exp\left(-\frac{\Delta t}{\tau_+}\right) & \text{if } \Delta t > 0 \quad (\text{LTP: A precedes B}) \\ -A_- \exp\left(\frac{\Delta t}{\tau_-}\right) & \text{if } \Delta t \leq 0 \quad (\text{LTD: A follows B}) \end{cases}$$

CEREBRUM STDPDiscretizer. Each call to `process(source_id, timestamp)` is a "spike". For every prior spike within `window_seconds`:

- 1) **LTP pass** — compute $\Delta t = t_{post} - t_{pre} > 0$ and increment $w_{pre \rightarrow post}$ by $A_+ \exp(-\Delta t / \tau_+)$.
- 2) **LTD pass** — simultaneously decrement the *anti-causal* direction $w_{post \rightarrow pre}$ by $A_- \exp(-\Delta t / \tau_-)$, clamped to 0.
- 3) **Forgetting** — after each spike, all weights are multiplied by $\lambda \in (0, 1]$ (`weight_decay`), providing exponential forgetting of old associations.

A directional `CAUSES` ($A \rightarrow B$) edge is emitted when:

$$w(A \rightarrow B) \geq w_{threshold} \quad \text{AND} \quad n(A \rightarrow B) \geq n_{min} \quad (22)$$

Default parameters ($A_+ = 0.1$, $A_- = 0.105$, $\tau_+ = \tau_- = 0.2s$) give slight LTD dominance — matching biological asymmetry measurements where depression is marginally stronger than potentiation to prevent runaway weight growth (Bi & Poo, 1998).

Biological correspondence.

STDP Biology	CEREBRUM STDPDiscretizer
Pre fires before post $\rightarrow$ LTP <i>seconds</i> $\rightarrow$ <i>w</i> increment	pre-spike within window
Post fires before pre $\rightarrow$ LTD	reverse direction decremented, clamped at 0
Exponential decay with $ \Delta t $	$A_{\pm} \exp(- \Delta t / \tau_{\pm})$
Multiplicative weight decay (forgetting)	<code>weight_decay</code> per spike
Synapse $\rightarrow$ "strengthened causal pathway"	<code>CAUSES</code> edge in graph
Threshold for synaptic consolidation	$w \geq w_{threshold}$, $n \geq n_{min}$

Integration with the reasoning graph. `CAUSES` edges participate in beam traversal identically to any other relation. In a manufacturing sensor graph, after hours of operation, CEREBRUM will have autonomously inferred directed causal chains — e.g., `Pump_A -[CAUSES]->Pressure_Spike_B -[CAUSES]->Relief_Valve_C` — from timing alone, without any domain-specific configuration or labeled training data.

12.9 REM Cycle — Graph Memory Consolidation (Phase 14)

Continuous operation accumulates speculative edges from stream discretization, STDP potentiation, and beam traversal. Without periodic consolidation, low-confidence edges degrade community structure and inflate beam noise. The REM Cycle (`core/rem_engine.py`) addresses this through three ordered phases.

Prune. Edges with confidence < 0.2 are candidates for removal. `BRIDGE_TWIN` edges are unconditionally immune — they represent structurally validated cross-community relays. All pruned edges are recorded in a rollback buffer.

Consolidate. DSCF community detection is re-run on the surviving graph, refreshing community assignments for subsequent CSA weight computation.

Synthesize. Entity pairs (u, v) not yet connected by an edge but satisfying:

$$\cos(\mathbf{e}_u, \mathbf{e}_v) \geq 0.8 \quad \text{and} \quad d_G(u, v) \leq 4 \quad (23)$$

receive a new `rem_synthesized` edge at confidence = 0.3. These edges make latent structural proximity explicit.

`dry_run=True` mode computes the proposed changes without modifying the graph. `rollback()` reverses the last

committed cycle (one-level undo), restoring pruned edges and removing synthesized ones.

API Endpoints

Method	Endpoint	Description
POST	/rem/run	Execute REM cycle (accepts <code>dry_run</code> param)
POST	/rem/rollback	Undo last REM cycle
GET	/rem/status	Last REMReport and rollback availability

Biological analog. The REM Cycle corresponds to NREM slow-wave sleep: the brain suppresses sensory input, prunes weakly-potentiated synapses (synaptic homeostasis), and consolidates frequently-activated pathways. CEREBRUM's Prune phase mirrors synaptic downscaling; Synthesize mirrors new associative link formation during consolidation.

12.10 InsightEngine — Eureka Moment Detection (Phase 15)

Not all reasoning paths carry equal informational value. A path that dramatically exceeds the historical score baseline for its community pair represents a structural discovery — an unexpected high-value connection the graph has not previously surfaced. The InsightEngine (`core/insight_engine.py`) detects, records, reinforces, and cements these events.

Surprise Signal. For a path P connecting community pair (c_i, c_j) :

$$\text{surprise}(P) = \text{score}(P) - \mu_{c_i, c_j} \quad (24)$$

where μ_{c_i, c_j} is the rolling mean score maintained in an $O(1)$ ring buffer. An `InsightEvent` is fired when $\text{surprise}(P) > \theta_{\text{salience}}$ (default 0.35). The composite score is:

$$\text{insight_score}(P) = \min\left(1, \frac{\text{surprise}(P) + \text{explanatory_power}(P)}{2}\right) \quad (25)$$

Three-Tier Architecture.

Tier	Mechanism	CPU Cost
Hot path	$O(1)$ ring buffer baseline update; immediate surprise check	0%
Warm path	Background daemon aggregating and ranking recent <code>InsightEvents</code>	0.01%
Cold path	Periodic full community boundary scan	50 ms/hr

The hot path runs synchronously in the traversal loop with no appreciable overhead. The warm daemon runs asynchronously and does not block reasoning. The cold scan runs hourly by default and is available on-demand via `POST /insight/scan`.

INSIGHT

*LINK*Edges. When an `InsightEvent` fires, the engine materializes an **INSIGHT_LINK** edge between the terminal entities of the

- **confidence = 0.85** — well above the REM prune threshold, making **INSIGHT_LINK** edges immune to REM pruning
- **weight = 2.0** — double the default, strongly boosting CSA attention for future traversals

Hebbian Reward Propagation. Every edge traversed by an insight path receives a weight increment:

$$\Delta w(u, v) = \delta_{\text{hebbian}} \times \text{insight_score}(P), \quad \text{capped at } 1.0 \quad (26)$$

This strengthens the entire chain of reasoning that produced the insight, not only the terminal connection.

REM + Insight Consolidation Loop.

- 1) **REM proposes** — Synthesize creates **rem_synthesized** edges at **confidence = 0.3**
- 2) **Traversal validates** — beam search uses synthesized edges; high-scoring paths generate surprise signals
- 3) **Insight cements** — `InsightEvent` materializes **INSIGHT_LINK** at **confidence = 0.85**, permanently above the REM prune threshold

Connections that never generate insight remain at 0.3 and are pruned in the next REM cycle. This implements a use-it-or-lose-it consolidation policy at the graph level.

API Endpoints

Method	Endpoint	Description
GET	/insight/status	InsightEngine status and recent events
GET	/insight/events	Recent <code>InsightEvents</code> (last n)
POST	/insight/scan	On-demand community boundary insight scan

Biological analogs.

Biological Mechanism	CEREBRUM InsightEngine
Dopamine prediction-error signal <i>score-rolling</i> <i>baseline</i>	Surprise = path
Neuroplasticity (LTP at insight synapses)	Hebbian reward propagation along insight path
Insight learning (aha moment) <i>treshold</i>	<code>InsightEvent</code> fired when surprise > salience
Long-term memory consolidation <i>LINKatconfidence = 0.85, resisting REM pruning</i>	INSIGHT
Sleep-dependent memory integration	REM proposes → Traversal validates → Insight cements

12.11 Insight Validation and Metacognition (Phase 16)

InsightValidator (`core/insight_validator.py`) applies two independent structural tests to each **InsightEvent** before its confidence is promoted.

Bilateral reverse traversal — the original graph (excluding the **INSIGHT_LINK** edge and the direct sourcetarget pair, to prevent circular self-validation) is searched for a return path from target to source within `max_hop` steps. A return path found through independent structure constitutes genuine confirmation:

$$\text{bilateral}(s, t) = \exists \text{ path}_{G \setminus \{\text{INSIGHT_LINK}, (s, t)\}}(t \rightarrow s, |P| \leq h_{\max}) \quad (27)$$

Multi-path corroboration — other nodes in the source community are tested as independent entry points to the same target. The source node is removed from the test graph so corroborating paths cannot simply route through the source’s existing edges:

$$\text{corroboration}(t) = \left| \left\{ u \in \mathcal{C}_s \setminus \{s, t\} \mid \exists \text{ path}_{G \setminus \{s\}}(u \rightarrow t, |P| \leq h_{\max}) \right\} \right| \quad (28)$$

Status	Condition	Confidence
corroborated	$\text{bilateral corroboration} \geq 2$	0.95
bilateral	$\text{bilateral corroboration} < 2$	0.92
unilateral	$\neg \text{bilateral corroboration} \geq 1$	0.85
isolated	$\neg \text{bilateral corroboration} = 0$	0.85 (flagged)

MetaInsightEngine (`core/meta_insight_engine.py`) maintains an **InsightGraph** — a directed graph where nodes are **InsightEvent** IDs and edges are structural relationships between pairs of insights. Four relationship types are scored for each new arrival relative to all prior events:

Relationship	Trigger	Score
chain	$A.\text{target} = B.\text{source}$ (or reverse)	$\frac{s_A + s_B}{2}$
shared_entity	Common node in entity sets	$\times 0.8$
community_overlap	Same community_leap ≥ 1	$\times 0.6$
temporal_cluster	$ T_A - T_B \leq \Delta t_{\text{window}}$	$\times 0.4$

When a relationship score exceeds the threshold, a **MetaInsightEvent** fires. When three insights form a chain $A \rightarrow B \rightarrow C$ in the **InsightGraph**, a depth-2 event fires:

$$A \xrightarrow{\text{meta}} B \xrightarrow{\text{meta}} C \Rightarrow \text{MetaInsightEvent}(\text{depth} = 2, \text{chain} = [A, B, C]) \quad (29)$$

This is second-order reasoning: the system observing a pattern in its own discovery history. The biological analog is episodic schema formation — the hippocampus binding related memories into abstract schemata during consolidation, and recognizing when two schemata are themselves related (the cognitive substrate of analogical reasoning).

Endpoint	Method	Description
POST	/insight/validate	Validate all pending InsightEvents
POST	/insight/validate/ insight_id	Validate one InsightEvent by ID
GET	/meta-insight/status	Retrieve engine state and InsightGraph size
GET	/meta-insight/event	Recent MetaInsightEvent list
GET	/meta-insight/graph	Full InsightGraph as JSON

1) Conclusion

We have presented CEREBRUM: a framework that enables Knowledge Graphs to reason using the structural principles of Transformer attention without training data, without an LLM, and with full interpretability.

The two core contributions — Community-Structured Attention (CSA) and Dual-Signal Community Fusion (DSCF) — work together to give a KG the dual character of multi-head attention: local cohesion (from DSCF’s LPA component) combined with global structural significance (from DSCF’s modularity component).

The resulting system produces reasoning paths, not Black-Box embeddings. Every answer is traceable to a sequence of verified graph edges. This architectural shift moves AI from probabilistic hidden-layer weights to a Glass-Box of deterministic paths — a vital transition in the modern AI/ML landscape. Every reasoning step names the community it traversed. This interpretability property, combined with the graph-grounded capability of graph-grounded inference, positions CEREBRUM as a meaningful complement to — and in certain domains, replacement for — LLM-based reasoning over structured knowledge.

The open questions identified in Section 10 define the ongoing research program. The benchmarks in Section 9 define the empirical standard. The architecture in Section 6 defines the core build. Phases 10–16 (Sections 11–12) demonstrate that the architecture scales to production, real-time streaming, experience-dependent structural plasticity, autonomous causal discovery, graph memory consolidation, structural insight detection, bilateral verification, and second-order metacognitive reasoning — all without modification to the core CSA or DSCF algorithms.

Phase 14’s REM Cycle implements a biologically-grounded consolidation loop: pruning noise, refreshing community structure, and synthesizing latent connections — the graph equivalent of NREM slow-wave sleep. Phase 15’s **InsightEngine** closes the loop: detecting when the graph discovers something genuinely unexpected, cementing those discoveries as high-confidence structural edges, and propagating Hebbian rewards along the reasoning chains that produced them. Together they give CEREBRUM a memory lifecycle — not just the ability to reason, but the ability to learn which reasoning mattered.

Phase 16 completes the cognitive architecture. **InsightValidator** ensures that discoveries are structurally verified before being trusted — requiring both a bilateral return path and independent corroboration from community peers, preventing any single speculative connection from

being self-certified. MetaInsightEngine adds second-order awareness: it watches the stream of verified discoveries and recognizes when multiple insights are themselves structurally related, firing depth-2 MetaInsightEvents for three-insight chains. The system now reasons, discovers, verifies its discoveries, and reasons about the pattern of its discoveries. This recursive self-awareness is the structural analog of analogical reasoning — the capacity to recognize that a new insight resembles a prior one, and that the resemblance itself is informative.

The name CEREBRUM refers to the optical phenomenon where two viewpoints on the same object yield depth perception that neither viewpoint alone provides. LPA and modularity are two viewpoints on the same graph. Their combination yields structural depth — attention heads with both short-range and long-range character — that neither produces alone. This multi-signal consensus is inspired by mid-level voting systems in triplex-redundant aircraft navigation, where the median value is selected to correct navigation errors. CEREBRUM applies this principle to "right the navigation errors" (hallucinations) of current language models by requiring structural consensus for every reasoning step.

That depth is what makes the KG reason.

References

- [1] Bi & Poo, "Synaptic Modifications in Cultured Hippocampal Neurons," *Journal of Neuroscience*, 1998.
- [2] Scarselli et al., "The Graph Neural Network Model," *IEEE TNNLS*, 2009. [2] Gilmer et al., "Neural Message Passing for Quantum Chemistry," *ICML*, 2017. [3] Velickovic et al., "Graph Attention Networks," *ICLR*, 2018. [4] Hamilton et al., "Inductive Representation Learning on Large Graphs," *NeurIPS*, 2017. [5] Bordes et al., "Translating Embeddings for Modeling Multi-relational Data (TransE)," *NeurIPS*, 2013. [6] Sun et al., "RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space," *ICLR*, 2019. [7] Xiong et al., "DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning," *EMNLP*, 2017. [8] Das et al., "Go for a Walk and Arrive at the Answer (MINERVA)," *ICLR*, 2018. [9] Yao et al., "KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using LLMs," 2023. [10] Chen et al., "KGPT: Knowledge-Grounded Pre-Training for Data-to-Text Generation," *EMNLP*, 2020. [11] Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *Microsoft Research*, 2024. [12] Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *ICLR*, 2024. [13] Blondel et al., "Fast Unfolding of Communities in Large Networks (Louvain)," *JSTAT*, 2008. [14] Traag et al., "From Louvain to Leiden: promoting Well-Connected Communities," *Scientific Reports*, 2019. [15] Raghavan et al., "Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks (LPA)," *Physical Review E*, 2007. [16] Galarraga et al., "AMIE: Association Rule Mining under Incomplete Evidence in Ontological Knowledge Bases," *WWW*, 2013.

REFERENCES